

■ LLM Bridge — Control en tiempo real del juego desde NEXUS

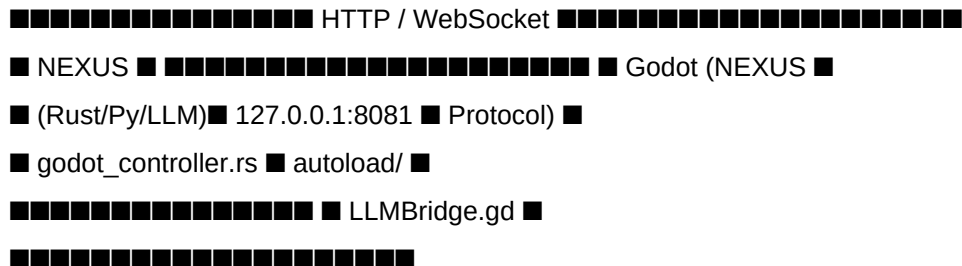
El **LLMBridge** convierte el motor Godot en un **cuerpo controlable por el LLM**.

Mientras el juego corre, el backend Rust/Python (y por tanto NEXUS) puede leer el estado del mundo, spawnear bestias, mover al jugador, inyectar daño/curación y ejecutar acciones en vivo.

El control del **Editor** de Godot se realiza a través del **Godot MCP** (157 herramientas).

Este puente controla el **juego en ejecución**.

Arquitectura



- **Servidor**: ``game/autoload/LLMBridge.gd`` — `autoload `NexusLLMBridge``, escucha en ``127.0.0.1``
- **Cliente**: ``src-tauri/src/godot_controller.rs`` — biblioteca `async` en Rust (`tokio` + `request`)
- **CLI de prueba**: ``src-tauri/src/godot_controller_cli.rs`` → binario ``godot-controller``.

Transportes

Transporte Uso

- ``GET /health`` Estado del puente
- ``GET /player`` Jugador: posición, vida, % vida
- ``GET /scene/tree`` Árbol de escena serializado (anidado, máx 12 niveles)
- ``POST /command`` Enviar un comando JSON `{cmd, args}`
- ``WS /nexus`` Mismo protocolo, conexión persistente bidireccional

Protocolo de comandos

Envías `{ "cmd": "<COMANDO>", "args": { ... } }` y recibes `{ "ok": true, "id": "...", "result"`

Comando Args Resultado

``PING`` — `{pong:true}`

``GET_PLAYER`` — info del jugador

``GET_SCENE_TREE`` — árbol de escena

``LIST_BEASTS`` — especies registradas

``SPAWN_BEAST`` `{species, x, z}` bestia creada

``KILL_BEASTS`` — bestias eliminadas

``DAMAGE_PLAYER`` `{amount}` daño aplicado

``HEAL_PLAYER`` `{amount}` cura aplicada

``MOVE_PLAYER`` `{x, z}` jugador teleportado

****Especies disponibles**:** ``lobo``, ``boar`` (jabali), ``spider`` (arana), ``bat`` (murcielago), ``golem``

Ejemplo HTTP

```
```bash
```

```
curl -X POST http://127.0.0.1:8081/command \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"cmd":"SPAWN_BEAST","args":{"species":"lobo","x":5,"z":-3}}'
```

```
```
```

Ejemplo WebSocket (Python)

```
```python
```

```
import websocket # pip install websocket-client
```

```
ws = websocket.create_connection("ws://127.0.0.1:8081/nexus")
```

```
ws.send('{"cmd":"GET_PLAYER","args":{}}')
```

```
print(ws.recv())
```

```
```
```

Cliente Rust

```
```rust
```

```

use godot_controller::GodotController;

let ctrl = GodotController::default();

// Leer el jugador
let player = ctrl.get_player().await?;

// Spawnear una bestia
let spawned = ctrl.spawn_beast("golem", 10.0, -5.0).await?;

// Infligir daño al jugador
let dmg = ctrl.damage_player(25.0).await?;
...

CLI de prueba

```bash
cargo run --bin godot-controller -- health
cargo run --bin godot-controller -- player
cargo run --bin godot-controller -- spawn lobo 5 -3
cargo run --bin godot-controller -- damage 25
cargo run --bin godot-controller -- heal 50
cargo run --bin godot-controller -- move 0 -10
...

---

```

Seguridad

- Escucha ****solo**** en `127.0.0.1` (loopback) — nunca expuesto a la red.
- Token de autenticación `AUTH\_TOKEN` definido en `LLMBridge.gd` (cambiar en producción).
 - Sin dependencias externas nuevas: GDScript puro + stdlib de Godot; Rust usa tokio/request ya

Validación

```

```bash

```

## Check sintáctico del puente (desde el directorio del juego)

```

../tools/godot/godot --headless --check-only --path game autoload/LLMBridge.gd

```

## Compilación del cliente Rust

```

cargo check --bin godot-controller
...

```